ReadyShell Architecture

End-to-end architecture for the current ReadyShell implementation in ReadyOS. This document is implementation-accurate for the local build and complements the generated overlay inventory report.

Keyboard note: on a C64 keyboard, users type `!` to enter the pipeline operator `|`.

SYSTEM SHAPE

ReadyShell is an overlayed C64 app with resident control code, a parse overlay, an execution-core overlay, and external command overlays. The resident shell owns orchestration; parsing and most execution live in the shared overlay window, and all eight overlays are preloaded into fixed REU slots during shell startup.

```
user types line
    |
    v
resident shell loop
    |
    +--> restore parser overlay from REU if needed
    |       parse source -> AST / runtime structures
    |
    +--> restore exec overlay (rsvm) from REU
            execute stages
              |
              +--> exec-core command
              `--> external command overlay call
                        |
                        +--> restore command overlay from REU
                        +--> run command
                        `--> restore rsvm from REU
```

Disk loading is now the startup preload path and the safety fallback. The normal runtime path restores overlays from REU.

RUNTIME MEMORY MODEL

REGION	RANGE	PURPOSE
Resident app window	`$1000–$C5FF`	ReadyShell-owned app RAM
Overlay load bytes	`$BDFE–$BDFF`	Overlay PRG load-address bytes
Overlay execution window	`$8E00–$C5FF`	Shared live overlay window
Resident BSS	`$870C–$8902`	Resident writable state
Resident heap	`$8904–$BDFD`	cc65 heap below overlay load
High runtime area	`$CA00–$CFFF`	Resident runtime state outside app snapshot

Resident growth reduces heap directly. Overlay growth does not reduce heap, but every overlay must still fit the `0x3800` window.

OVERLAY SET

OVERLAY	FILE	ROLE	LOADING
1	`rsparser.prg`	lexer, parser, parse support	boot-preloaded, bank `0x40`
2	`rsvm.prg`	values, vars, pipes, formatting, shared execution	boot-preloaded, bank `0x40`
3	`rsdrvilst.prg`	`DRVI`, `LST`	boot-preloaded, bank `0x40`
4	`rsldv.prg`	`LDV`	boot-preloaded, bank `0x41`
5	`rsstv.prg`	`STV`	boot-preloaded, bank `0x40`
6	`rsfops.prg`	`DEL`, `REN`, `PUT`, `ADD`	boot-preloaded, bank `0x41`
7	`rscat.prg`	`CAT`	boot-preloaded, bank `0x41`
8	`rscopy.prg`	`COPY`	boot-preloaded, bank `0x41`

COMMAND FAMILIES

- Exec-core commands in `OVERLAY2`: `PRT`, `MORE`, `TOP`, `SEL`, `GEN`, `TAP`
- External commands in overlays `3-8`: `DRVI`, `LST`, `LDV`, `STV`, `DEL`, `REN`, `PUT`, `ADD`, `CAT`, `COPY`

External commands are driven through one generic resident execution path plus the REU-backed command registry.

LST

Directory listing now takes more than a bare drive number.

- Pattern strings support CBM wildcards such as `*` and `?`.
- Type filters accept comma-separated tokens such as `PRG`, `SEQ`, `USR`, `REL`, `DIR`, `CBM`, and `DEL`.
- Examples: `LST "t*", 9, "PRG,SEQ"`, `LST "*,PRG", "PRG"`.

LDV

Load-value syntax matches the filename-oriented commands now.

- Embedded drive form: `LDV "9:snap"`
- Separate drive form: `LDV "snap", 9`
- Embedded drive prefixes win if both are present.

STV

Store-value syntax mirrors `LDV`.

- Embedded drive form: `STV $A, "9:snap"`
- Separate drive form: `STV $A, "snap", 9`
- The same precedence rule applies: embedded prefix beats fallback drive.

OTHER FILE COMMANDS

Filename handling is intentionally more uniform across the overlay commands.

- `CAT`, `PUT`, `ADD`, `DEL`, and `COPY` honor embedded drive prefixes.
- `REN` still uses its traditional explicit drive argument form.
- This keeps overlay-local parsing behavior aligned with the broader shell UX.

The practical rule is simple: if a command accepts a filename and an embedded drive prefix makes sense for it, ReadyShell now prefers that form and treats a trailing numeric drive as a fallback selector.

PIPELINE AND PARSE FLOW

```
source line
    |
    v
OVERLAY1: lexer + parser
    |
    v
resident regains control
    |
    v
OVERLAY2: execute pipeline stage-by-stage
    |
    +--> expr stage
    +--> filter stage
    +--> foreach stage
    +--> exec-core command
    `--> external command
```

The overlay command ABI defines `BEGIN`, `ITEM`, `RUN`, `PROCESS`, and `END`. The current shipped commands use `RUN` for most command overlays and `BEGIN + ITEM` for `CAT`.

EXTERNAL COMMAND REGISTRY

REGION	RANGE	PURPOSE
Registry header	`$488010–$488017`	magic, version, counts
Command descriptors	`$488020–$48807F`	external command metadata
Overlay state table	`$48808D–$4880EB`	overlay phase, cache bank, slot, and load state

The registry is stored in REU metadata rather than resident heap/BSS, so new external commands do not require resident per-command wrapper growth.

EXTERNAL COMMAND CALL FLOW

```
OVERLAY2 decides command is external
    |
    v
resident vm_cmd_external()
    |
    v
lookup descriptor in REU registry
    |
    v
prepare external overlay from overlay-state record
    |
    +--> normal case: restore full slot from REU
    `--> fallback: load from disk and refresh cache slot
    |
    v
call one overlay dispatcher with handler id
    |
    v
command overlay runs command body
    |
    v
resident restores OVERLAY2 from REU
```

Repeating `LST` now repeats the REU restore, not a disk load. The same is true for file commands such as `COPY "8:notes","9:notes"`.

REU USAGE

```
bank 0x40

+-----------------------------------------+ $400000
| OVERLAY1 parse slot                     |
+-----------------------------------------+ $403800
| OVERLAY2 exec slot                      |
+-----------------------------------------+ $407000
| OVERLAY3 rsdrvilst slot                 |
+-----------------------------------------+ $40A800
| OVERLAY5 rsstv slot                     |
+-----------------------------------------+ $40E000
| free tail (8192 bytes)                  |
+-----------------------------------------+ $40FFFF

bank 0x41

+-----------------------------------------+ $410000
| OVERLAY4 rsldv slot                     |
+-----------------------------------------+ $413800
| OVERLAY6 rsfops slot                    |
+-----------------------------------------+ $417000
| OVERLAY7 rscat slot                     |
+-----------------------------------------+ $41A800
| OVERLAY8 rscopy slot                    |
+-----------------------------------------+ $41E000
| free tail (8192 bytes)                  |
+-----------------------------------------+ $41FFFF
```

These are full-window snapshots, not just file payloads, so writable overlay data survives switching.

SHARED STATE BANKS

```
bank 0x43
  debug/probe ring and verification bytes

bank 0x48
  command scratch      $480000-$487FFF
  command registry     $488010-$48807F
  shared metadata      $48808D-$4880FB
  pause flag           $4880FC
  REU heap metadata    $488000-$4880FF
  REU value arena      $488100-$48FEFF
```

RESPONSIBILITIES

- Resident code owns the shell loop, overlay boot/load/restore logic, generic external command dispatch, screen/platform glue, and REU coordination.
- Overlay 1 owns lexing, parsing, parse support, and cleanup.
- Overlay 2 owns runtime values, variables, formatting, pipes, command lookup, capability classification, and shared execution helpers.
- Command overlays own command-specific disk logic, serialization logic, and overlay-local dispatchers.

CURRENT CONSTRAINTS

- Resident heap below the overlay load address is `1274` bytes.
- `OVERLAY2` is still the tightest overlay and remains the next likely growth bottleneck.
- Each fixed cache bank leaves `8192` bytes of free tail after the current slot layout.
- External commands now pay a one-time startup preload cost instead of repeated disk loads during each command call.